CIT4001 SOFTWARE IMPLEMENTATION AND TESTING

Testing and control flow appendix

PracticeChart EHR

University of Technology, Jamaica

Group 2

This document describes the implemented course application and the evidence recorded for it. It uses fictional patient data. Human training and user acceptance remain pending.

Public course copy. Student identification numbers and private course instruments are excluded.

Prepared for group review. This is not a record of clinical approval, a live pilot, completed training, or completed acceptance testing.

Strategy and environment

Test the database and domain services first. Integrate the server routes and browser interface after those checks. Use fictional fixtures and a separate local test database. Use the complete application for system tests. Use manual fault cases for missing files, invalid data, stale changes, and rejected access. Keep regression checks on the protected paths.

Vitest covers partitions, boundaries, and domain/database outcomes. Playwright covers real browser workflows and direct route restrictions. Axe checks selected pages. The restore drill tests a full encrypted recovery bundle. The performance script states its load, timing definitions, environment, and limits.

This is a bottom-up integration strategy: validation and persistence, then service transactions, then routes, then role workflows. Database fixtures and test callers act as drivers. File failure tests isolate storage. No stub is presented as proof that a real database constraint works. The booking tests use actual PostgreSQL transactions.

Feasible independent path proof cases

These cases take different application decision edges. Expected and actual outcomes are checked by the named executable tests. Graph node numbers are in CONTROL_FLOW.md and the per-function graphs. The generated syntactic path lists are not labelled as executed paths merely because a graph tool found them.

Module	Inputs and starting state	Expected outcome	Executable evidence
Login	Valid active doctor and correct password	Create session; token resolves to doctor	logs in and stores only a token hash
Login	Wrong password or missing username	Same rejection; no new valid session	rejects wrong and missing accounts with the same error
Login	Eight failed attempts in the window, then another	Reject with 429	enforces the login attempt limit
Session	Expired token or removed session	No authenticated account	rejects an expired session; token deletion assertion
Patient save	Valid new person	Commit patient and unique MRN	preserves distinct MRNs
Patient save	Existing record and current version	Commit demographic change	permits one of two stale demographic changes
Patient save	Two writers use one expected version	One save; one conflict	stale demographic race test
Booking	Active patient and free interval	Commit confirmed booking	browser confirmation workflow
Booking	Same doctor, overlapping interval	Reject and preserve other booking	adjacent and overlap test
Booking	Two requests claim one interval	Exactly one commits	simultaneous booking test
Booking	Start outside hours or end beyond closing	Reject	hour boundary test
Reschedule	Current confirmed booking, target conflicts	Roll back; original time remains	keeps the original booking when a reschedule conflicts

Module	Inputs and starting state	Expected outcome	Executable evidence
Cancel	Current confirmed booking	Keep row, cancel, release slot	releases a cancelled slot and retains its history
Visit	No final diagnosis	Reject filing	clinical correction and filing test
Clinical save	New visit, valid draft	Append draft revision	preserves revisions test
Clinical save	Stale expected revision	Reject with conflict	stale revision assertion
Clinical save	Previous final entry, no reason	Reject correction	correction reason assertion
Clinical save	Previous final entry, final correction and reason	Append revision; keep earlier content	correction test and complete browser workflow
Prescription	Doctor enters explicit medicine fields	Store and print those fields	medication storage test and browser prescription check
File retrieval	Approved path and existing bytes	Return exact bytes privately	file storage test and browser retrieval test
File retrieval	Unsafe path or missing object	Reject or return unavailable	private path and missing-file tests
Invoice	Valid quantities and exact unit cents	Calculate exact total; create draft	total calculation and invoice state test
Invoice	Issue current draft	Mark issued	invoice state and browser issue test
Invoice	Void without reason	Reject	void reason assertion
Invoice	Void issued invoice with reason	Keep row and mark void	invoice state test
Account	Front desk requests account administration	Reject and audit denied access	front desk account creation and direct URL tests
Account	Disable the final doctor	Reject in PostgreSQL	last active doctor test
Account	Disable another test account	Revoke sessions and deny login	account access test
History	Update or delete protected rows	Reject	immutable clinical and audit tests
Recovery	Complete encrypted dump and matching file	Restore into isolated database	restore-result.json

Partitions and boundaries

Use valid and invalid dates, leap days, today's birth date, future dates, required names, maximum text lengths, invalid phone characters, invalid roles, and password length limits. No patient age range is invented.

Use free, occupied, adjacent, blocked, outside-hours, cancelled, and stale appointment states. Test opening time, one minute before opening, exact closing time, and an interval that starts at closing. Test zero, negative, valid, and excessive durations. Use database races for simultaneous requests.

Use empty, approved, wrong-signature, mismatched MIME, missing, maximum-size, and too-large files. Test public demo sample restrictions. Use zero, negative, fractional, valid, and excessive invoice values. Exact cents

avoid floating-point totals in stored invoices.

Use valid sessions, wrong passwords, unknown accounts, disabled accounts, expired sessions, front desk direct clinical requests, guessed document IDs, and forged origins. A hidden button alone is not evidence of a permission check.

Coverage and feasibility limits

The V8 coverage summary applies to the tested library files and the Vitest process. Browser work is measured separately. It is not valid to add the percentages together. Full statement, branch, and possible-path coverage are not claimed.

Graph candidates can include contradictory state assumptions. Examples include saving successfully after the same execution has raised a stale-version error, treating one previous revision as both draft and final, or filing a visit that is both open and ready. Those combinations are infeasible. A loop-skipping path also needs an empty collection. A medicine loop path needs the corresponding number of medicines. Documented tests use feasible records and states.

Release criteria

Type checking and the production build must pass. Unit, integration, and complete browser workflows must pass. Role and private file failures must be fixed before release. The full recovery test must pass. Scan Git contents for secrets and private source material. Record load results and axe results. Record any unmet long-term target as a limit.

Human acceptance, actual staff training, legal review, and testing on a real phone remain separate tasks. Blank forms are supplied. No automated result is presented as completed human testing.

The generator reads the implemented TypeScript functions. It expands transaction callbacks. It does not expand database engines or library internals. SQL constraints have separate integration tests. Graphs use one entry and one exit. Compound conditions are split. Each explicit I/O try block has one fault branch.

Face counts use a planar half-edge traversal. Parallel directed edges add a lens face. Predicate counts are taken from nodes with two outgoing edges. The third method uses the separate edge and node counts. These three results must agree.

Function	Nodes N	Edges E	Faces R	Predicates P	R	P + 1	E - N + 2
patientList	7	6	1	0	1	1	1
patientDetail	14	14	2	1	2	2	2
savePatient	17	18	3	2	3	3	3
archivePatient	13	14	3	2	3	3	3
reconcilePatient	21	25	6	5	6	6	6
schedule	10	11	3	2	3	3	3
availability	7	7	2	1	2	2	2

Function	Nodes N	Edges E	Faces R	Predicates P	R	P + 1	E - N + 2
bookAppointment	20	23	5	4	5	5	5
appointmentStatus	10	10	2	1	2	2	2
startVisit	19	23	6	5	6	6	6
saveClinical	29	37	10	9	10	10	10
fileVisit	17	21	6	5	6	6	6
saveInvoice	10	9	1	0	1	1	1
setInvoiceStatus	15	18	5	4	5	5	5
invoiceList	4	3	1	0	1	1	1
manageUser	33	40	9	8	9	9	9
changePassword	17	19	4	3	4	4	4
dashboard	9	8	1	0	1	1	1
reportData	24	31	9	8	9	9	9
requireRole	5	5	2	1	2	2	2
getSession	8	9	3	2	3	3	3
login	23	27	6	5	6	6	6
storeFile	23	31	10	9	10	10	10
readPrivateFile	16	21	7	6	7	7	7
invoiceTotal	7	8	3	2	3	3	3
fileMime	11	15	6	5	6	6	6

Basis path candidates

The following paths are independent in the generated graph. They are syntactic candidates, not a claim that every candidate is feasible or was executed. Branch conditions can depend on earlier state. For example, a stale revision cannot also reach a successful save in the same execution. Repeated file and medication paths depend on the number of records. Do not invent inputs to force an impossible path.

Actual feasible scenarios and test outcomes are mapped in TEST_PLAN.md. Full possible-path coverage is not claimed. V8 statement and branch coverage is recorded separately. A loop can have more possible executions than its basis-path count.

patientList

Source: src/lib/service.ts:46. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 5 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 5 -> 2 -> 5 -> 7 -> 6 -> 4 -> 3. Additional parallel-edge lens faces: 0.

patientDetail

Source: src/lib/service.ts:56. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 5 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 12 -> 13 -> 14 -> 5 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 12 -> 13 -> 14 -> 5 -> 11 -> 10 -> 9 -> 8 -> 7 -> 6 -> 4 -> 3; 2 -> 5 -> 14 -> 13 -> 12 -> 10 -> 11 -> 5. Additional parallel-edge lens faces: 0.

savePatient

Source: src/lib/service.ts:98. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 12 -> 13 -> 5 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 6 -> 14 -> 15 -> 16 -> 17 -> 5 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 14 -> 15 -> 16 -> 17 -> 5 -> 2 -> 11 -> 10 -> 9 -> 8 -> 7 -> 6 -> 4 -> 3; 2 -> 5 -> 13 -> 12 -> 10 -> 11; 5 -> 17 -> 16 -> 15 -> 14 -> 6 -> 7 -> 8 -> 9 -> 10 -> 12 -> 13. Additional parallel-edge lens faces: 0.

archivePatient

Source: src/lib/service.ts:131. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 10 -> 11 -> 12 -> 6 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 10 -> 13 -> 11 -> 12 -> 6 -> 2

Face boundaries: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 10 -> 11 -> 12 -> 6 -> 2 -> 9 -> 8 -> 7 -> 5 -> 4 -> 3; 2 -> 6 -> 12 -> 11 -> 13 -> 10 -> 8 -> 9; 10 -> 13 -> 11. Additional parallel-edge lens faces: 0.

reconcilePatient

Source: src/lib/service.ts:149. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10 -> 11 -> 12 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 16 -> 17 -> 18 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 9 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 16 -> 18 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 18 -> 2

Face boundaries: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10 -> 11 -> 12 -> 2 -> 7 -> 6 -> 5 -> 4 -> 3; 2 -> 12 -> 11 -> 13 -> 14 -> 15 -> 18; 2 -> 9 -> 21 -> 20 -> 19 -> 17 -> 16 -> 15 -> 14 -> 13 -> 11 -> 10 -> 8 -> 6 -> 7; 2 -> 18 -> 17 -> 19 -> 20 -> 21 -> 9; 15 -> 16 -> 18; 16 -> 17 -> 18. Additional parallel-edge lens faces: 0.

schedule

Source: src/lib/service.ts:189. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 7 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 10 -> 5 -> 6 -> 8 -> 9 -> 7 -> 2

- Candidate 3: 1 -> 3 -> 5 -> 6 -> 8 -> 9 -> 7 -> 2

Face boundaries: 1 -> 3 -> 5 -> 4 -> 3; 2 -> 7 -> 9 -> 8 -> 6 -> 5 -> 10 -> 4 -> 5 -> 6 -> 8 -> 9 -> 7; 3 -> 4 -> 10 -> 5. Additional parallel-edge lens faces: 0.

availability

Source: src/lib/service.ts:204. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 2 -> 5 -> 4 -> 3; 2 -> 7 -> 6 -> 4 -> 5. Additional parallel-edge lens faces: 0.

bookAppointment

Source: src/lib/service.ts:217. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 5 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 18 -> 19 -> 20 -> 5 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 8 -> 7 -> 6 -> 4 -> 3; 2 -> 15 -> 14 -> 16 -> 17 -> 5; 2 -> 5 -> 20 -> 19 -> 18 -> 10 -> 8 -> 9; 2 -> 9 -> 7 -> 8 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15; 5 -> 17 -> 16 -> 14 -> 13 -> 12 -> 11 -> 10 -> 18 -> 19 -> 20. Additional parallel-edge lens faces: 0.

appointmentStatus

Source: src/lib/service.ts:270. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 5 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 5 -> 2 -> 8 -> 7 -> 6 -> 4 -> 3; 2 -> 5 -> 10 -> 9 -> 7 -> 8. Additional parallel-edge lens faces: 0.

startVisit

Source: src/lib/service.ts:302. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 5 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 11 -> 12 -> 13 -> 15 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 11 -> 12 -> 15 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 17 -> 18 -> 19 -> 5 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 10 -> 11 -> 12 -> 15 -> 2 -> 8 -> 7 -> 6 -> 4 -> 3; 2 -> 15 -> 14 -> 16 -> 17 -> 18 -> 19 -> 5; 2 -> 5 -> 19 -> 18 -> 17 -> 10 -> 9 -> 7 -> 8; 10 -> 17 -> 16 -> 14 -> 13 -> 12 -> 11; 12 -> 13 -> 15; 13 -> 14 -> 15. Additional parallel-edge lens faces: 0.

saveClinical

Source: src/lib/service.ts:348. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 10 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 15 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 20 -> 21 -> 22 -> 23 -> 24 -> 23 -> 25 -> 26 -> 27 -> 7 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 20 -> 21 -> 22 -> 23 -> 24 -> 23 -> 25 -> 29 -> 26 -> 27 -> 7 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 20 -> 21 -> 22 -> 23 -> 25 -> 26 -> 27 -> 7 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 20 -> 21 -> 22 -> 25 -> 26 -> 27 -> 7 -> 2

- Candidate 7: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 20 -> 28 -> 21 -> 22 -> 23 -> 24 -> 23 -> 25 -> 26 -> 27 -> 7 -> 2

- Candidate 8: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 14 -> 16 -> 17 -> 18 -> 19 -> 21 -> 22 -> 23 -> 24 -> 23 -> 25 -> 26 -> 27 -> 7 -> 2

- Candidate 9: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 15 -> 2

- Candidate 10: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 16 -> 17 -> 18 -> 19 -> 20 -> 21 -> 22 -> 23 -> 24 -> 23 -> 25 -> 26 -> 27 -> 7 -> 2

Face boundaries: 1 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 11 -> 12 -> 13 -> 15 -> 2 -> 10 -> 9 -> 8 -> 6 -> 5 -> 4 -> 3; 2 -> 15 -> 14 -> 16 -> 17 -> 18 -> 19 -> 21 -> 22 -> 25 -> 29 -> 26 -> 27 -> 7; 2 -> 7 -> 27 -> 26 -> 25 -> 23 -> 22 -> 21 -> 20 -> 19 -> 18 -> 17 -> 16 -> 12 -> 11 -> 9 -> 10; 12 -> 16 -> 14 -> 13; 13 -> 14 -> 15; 19 -> 20 -> 28 ->

21; 20 -> 21 -> 28; 22 -> 23 -> 24 -> 23 -> 25; 25 -> 26 -> 29. Additional parallel-edge lens faces: 1.

fileVisit

Source: src/lib/service.ts:410. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 13 -> 14 -> 15 -> 16 -> 17 -> 5 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 13 -> 14 -> 16 -> 17 -> 5 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 12 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 12 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 12 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 12 -> 11 -> 10 -> 9 -> 8 -> 7 -> 6 -> 4 -> 3; 2 -> 5 -> 17 -> 16 -> 15 -> 14 -> 13 -> 11 -> 12; 2 -> 12 -> 10 -> 11 -> 13 -> 14 -> 16 -> 17 -> 5; 8 -> 9 -> 12; 9 -> 10 -> 12; 14 -> 15 -> 16. Additional parallel-edge lens faces: 0.

saveInvoice

Source: src/lib/service.ts:447. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 6 -> 2

Face boundaries: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 6 -> 2 -> 6 -> 10 -> 9 -> 8 -> 7 -> 5 -> 4 -> 3. Additional parallel-edge lens faces: 0.

setInvoiceStatus

Source: src/lib/service.ts:465. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 7 -> 9 -> 10 -> 11 -> 12 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 5 -> 7 -> 9 -> 10 -> 11 -> 13 -> 14 -> 8 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 5 -> 7 -> 9 -> 15 -> 10 -> 11 -> 12 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 7 -> 9 -> 10 -> 11 -> 12 -> 2

Face boundaries: 1 -> 3 -> 4 -> 7 -> 9 -> 10 -> 11 -> 12 -> 2 -> 6 -> 5 -> 4 -> 3; 2 -> 12 -> 11 -> 13 -> 14 -> 8; 2 -> 8 -> 14 -> 13 -> 11 -> 10 -> 15 -> 9 -> 7 -> 5 -> 6; 4 -> 5 -> 7; 9 -> 15 -> 10. Additional parallel-edge lens faces: 0.

invoiceList

Source: src/lib/service.ts:500. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 2

Face boundaries: 1 -> 3 -> 4 -> 2 -> 4 -> 3. Additional parallel-edge lens faces: 0.

manageUser

Source: src/lib/service.ts:506. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> 10 -> 11 -> 12 -> 13 -> 8 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> 10 -> 12 -> 13 -> 8 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 22 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 23 -> 24 -> 25 -> 26 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 23 -> 24 -> 25 -> 27 -> 28 -> 29 -> 30 -> 31 -> 18 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 23 -> 24 -> 25 -> 27 -> 29 -> 30 -> 31 -> 18 -> 2

- Candidate 7: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 23 -> 33 -> 24 -> 25 -> 26 -> 2

- Candidate 8: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 23 -> 24 -> 25 -> 26 -> 2

- Candidate 9: 1 -> 3 -> 4 -> 14 -> 15 -> 32 -> 16 -> 17 -> 19 -> 20 -> 21 -> 22 -> 2

Face boundaries: 1 -> 3 -> 4 -> 14 -> 15 -> 16 -> 17 -> 19 -> 20 -> 21 -> 22 -> 2 -> 8 -> 13 -> 12 -> 11 -> 10 -> 9 -> 7 -> 6 -> 5 -> 4 -> 3; 2 -> 22 -> 21 -> 23 -> 33 -> 24 -> 25 -> 27 -> 29 -> 30 -> 31 -> 18; 2 -> 26 -> 25 -> 24 -> 23 -> 20 -> 19 -> 17 -> 16 -> 32 -> 15 -> 14 -> 4 -> 5 -> 6 -> 7 -> 9 -> 10 -> 12 -> 13 -> 8; 2 -> 18 -> 31 -> 30 -> 29 -> 28 -> 27 -> 25 -> 26; 10 -> 11 -> 12; 15 -> 32 -> 16; 20 -> 23 -> 21; 23 -> 24 -> 33; 27 -> 28 -> 29. Additional parallel-edge lens faces: 0.

changePassword

Source: src/lib/service.ts:567. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 11 -> 12 -> 14 -> 15 -> 16 -> 13 -> 17 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 7 -> 8 -> 9 -> 10 -> 2

Face boundaries: 1 -> 3 -> 4 -> 7 -> 8 -> 9 -> 10 -> 2 -> 6 -> 5 -> 4 -> 3; 2 -> 10 -> 9 -> 11 -> 12 -> 14 -> 15 -> 16 -> 13 -> 17; 2 -> 17 -> 13 -> 16 -> 15 -> 14 -> 12 -> 11 -> 9 -> 8 -> 7 -> 5 -> 6; 4 -> 5 -> 7. Additional parallel-edge lens faces: 0.

dashboard

Source: src/lib/service.ts:591. Graph.

- Candidate 1: 1 -> 3 -> 5 -> 6 -> 7 -> 8 -> 9 -> 4 -> 2

Face boundaries: 1 -> 3 -> 5 -> 6 -> 7 -> 8 -> 9 -> 4 -> 2 -> 4 -> 9 -> 8 -> 7 -> 6 -> 5 -> 3. Additional parallel-edge lens faces: 0.

reportData

Source: src/lib/service.ts:609. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 2

- Candidate 2: 1 -> 3 -> 5 -> 6 -> 7 -> 8 -> 2

- Candidate 3: 1 -> 3 -> 5 -> 6 -> 7 -> 9 -> 10 -> 11 -> 2

- Candidate 4: 1 -> 3 -> 5 -> 6 -> 7 -> 9 -> 12 -> 13 -> 14 -> 15 -> 2

- Candidate 5: 1 -> 3 -> 5 -> 6 -> 7 -> 9 -> 12 -> 13 -> 14 -> 16 -> 17 -> 2

- Candidate 6: 1 -> 3 -> 5 -> 6 -> 7 -> 9 -> 12 -> 13 -> 14 -> 16 -> 18 -> 19 -> 20 -> 2

- Candidate 7: 1 -> 3 -> 5 -> 6 -> 7 -> 9 -> 12 -> 13 -> 14 -> 16 -> 18 -> 19 -> 21 -> 22 -> 23 -> 2

- Candidate 8: 1 -> 3 -> 5 -> 6 -> 7 -> 9 -> 12 -> 13 -> 14 -> 16 -> 18 -> 19 -> 21 -> 22 -> 24 -> 2

- Candidate 9: 1 -> 3 -> 5 -> 7 -> 8 -> 2

Face boundaries: 1 -> 3 -> 5 -> 7 -> 8 -> 2 -> 4 -> 3; 2 -> 8 -> 7 -> 9 -> 12 -> 13 -> 14 -> 16 -> 18 -> 19 -> 21 -> 22 -> 24; 2 -> 11 -> 10 -> 9 -> 7 -> 6 -> 5 -> 3 -> 4; 2 -> 15 -> 14 -> 13 -> 12 -> 9 -> 10 -> 11; 2 -> 17 -> 16 -> 14 -> 15; 2 -> 20 -> 19 -> 18 -> 16 -> 17; 2 -> 23 -> 22 -> 21 -> 19 -> 20; 2 -> 24 -> 22 -> 23; 5 -> 6 -> 7. Additional parallel-edge lens faces: 0.

requireRole

Source: src/lib/auth.ts:24. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 2

- Candidate 2: 1 -> 3 -> 2

Face boundaries: 1 -> 3 -> 4 -> 5 -> 2 -> 3; 2 -> 5 -> 4 -> 3. Additional parallel-edge lens faces: 0.

getSession

Source: src/lib/auth.ts:30. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 2

- Candidate 2: 1 -> 3 -> 5 -> 6 -> 7 -> 2

- Candidate 3: 1 -> 3 -> 5 -> 6 -> 8 -> 7 -> 2

Face boundaries: 1 -> 3 -> 5 -> 6 -> 7 -> 2 -> 4 -> 3; 2 -> 7 -> 8 -> 6 -> 5 -> 3 -> 4; 6 -> 8 -> 7. Additional parallel-edge lens faces: 0.

login

Source: src/lib/auth.ts:54. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 16 -> 17 -> 19 -> 20 -> 21 -> 18 -> 22 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 14 -> 15 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 11 -> 14 -> 15 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 23 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 2

Face boundaries: 1 -> 3 -> 4 -> 5 -> 7 -> 8 -> 9 -> 10 -> 11 -> 14 -> 15 -> 2 -> 6 -> 5 -> 4 -> 3; 2 -> 15 -> 14 -> 13 -> 16 -> 17 -> 19 -> 20 -> 21 -> 18 -> 22; 2 -> 22 -> 18 -> 21 -> 20 -> 19 -> 17 -> 16 -> 13 -> 12 -> 11 -> 10 -> 23 -> 9 -> 8 -> 7 -> 5 -> 6; 9 -> 23 -> 10; 11 -> 12 -> 14; 12 -> 13 -> 14. Additional parallel-edge lens faces: 0.

storeFile

Source: src/lib/storage.ts:31. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 9 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 12 -> 2

- Candidate 4: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 16 -> 21 -> 2

- Candidate 5: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 17 -> 18 -> 19 -> 20 -> 21 -> 2

- Candidate 6: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 13 -> 14 -> 15 -> 17 -> 23 -> 18 -> 19 -> 20 -> 21 -> 2

- Candidate 7: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 11 -> 13 -> 22 -> 14 -> 15 -> 16 -> 21 -> 2

- Candidate 8: 1 -> 3 -> 4 -> 6 -> 7 -> 8 -> 10 -> 13 -> 14 -> 15 -> 16 -> 21 -> 2

- Candidate 9: 1 -> 3 -> 4 -> 6 -> 7 -> 9 -> 2

- Candidate 10: 1 -> 3 -> 5 -> 2

Face boundaries: 1 -> 3 -> 5 -> 4 -> 3; 2 -> 9 -> 8 -> 10 -> 13 -> 22 -> 14 -> 15 -> 17 -> 18 -> 19 -> 20 -> 21; 2 -> 12 -> 11 -> 10 -> 8 -> 7 -> 6 -> 4 -> 5; 2 -> 21 -> 16 -> 15 -> 14 -> 13 -> 11 -> 12; 2 -> 5 -> 3 -> 4 -> 6 -> 7 -> 9; 7 -> 8 -> 9; 10 -> 11 -> 13; 13 -> 14 -> 22; 15 -> 16 -> 21 -> 20 -> 19 -> 18 -> 23 -> 17; 17 -> 23 -> 18. Additional parallel-edge lens faces: 0.

readPrivateFile

Source: src/lib/storage.ts:67. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 2

- Candidate 2: 1 -> 3 -> 5 -> 6 -> 15 -> 2

- Candidate 3: 1 -> 3 -> 5 -> 7 -> 8 -> 9 -> 10 -> 11 -> 6 -> 15 -> 2

- Candidate 4: 1 -> 3 -> 5 -> 7 -> 8 -> 9 -> 10 -> 12 -> 2

- Candidate 5: 1 -> 3 -> 5 -> 7 -> 8 -> 9 -> 11 -> 6 -> 15 -> 2

- Candidate 6: 1 -> 3 -> 5 -> 7 -> 13 -> 14 -> 2

- Candidate 7: 1 -> 3 -> 5 -> 7 -> 13 -> 16 -> 14 -> 2

Face boundaries: 1 -> 3 -> 5 -> 6 -> 15 -> 2 -> 4 -> 3; 2 -> 12 -> 10 -> 9 -> 8 -> 7 -> 13 -> 16 -> 14; 2 -> 15 -> 6 -> 11 -> 10 -> 12; 2 -> 14 -> 13 -> 7 -> 5 -> 3 -> 4; 5 -> 7 -> 8 -> 9 -> 11 -> 6; 9 -> 10 -> 11; 13 -> 14 -> 16. Additional parallel-edge lens faces: 0.

invoiceTotal

Source: src/lib/validation.ts:94. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 5 -> 6 -> 2

- Candidate 2: 1 -> 3 -> 4 -> 5 -> 7 -> 2

- Candidate 3: 1 -> 3 -> 4 -> 6 -> 2

Face boundaries: 1 -> 3 -> 4 -> 6 -> 5 -> 4 -> 3; 2 -> 7 -> 5 -> 6; 2 -> 6 -> 4 -> 5 -> 7. Additional parallel-edge lens faces: 0.

fileMime

Source: src/lib/validation.ts:125. Graph.

- Candidate 1: 1 -> 3 -> 4 -> 2

- Candidate 2: 1 -> 3 -> 5 -> 6 -> 2

- Candidate 3: 1 -> 3 -> 5 -> 7 -> 8 -> 9 -> 10 -> 2

- Candidate 4: 1 -> 3 -> 5 -> 7 -> 8 -> 9 -> 11 -> 2

- Candidate 5: 1 -> 3 -> 5 -> 7 -> 8 -> 11 -> 2

- Candidate 6: 1 -> 3 -> 5 -> 7 -> 11 -> 2

Face boundaries: 1 -> 3 -> 5 -> 6 -> 2 -> 4 -> 3; 2 -> 6 -> 5 -> 7 -> 11; 2 -> 10 -> 9 -> 8 -> 7 -> 5 -> 3 -> 4; 2 -> 11 -> 9 -> 10; 7 -> 8 -> 11; 8 -> 9 -> 11. Additional parallel-edge lens faces: 0.

Generated from the current application source. The source listing remains the detailed programme reference. SQL and library internals are represented as calls. Transaction callbacks are expanded. Array callbacks stay within their library operation. The graphs cover explicit application branches, short-circuit conditions, conditional expressions, and loops. A try block includes an explicit I/O failure branch. External library internals are not counted.

patientList

Search demographic records with parameters and record the search event.

Source: src/lib/service.ts. Graph: patientList.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START patientList
  GO TO L3
L2: END PROCEDURE
L3: const term = `%${search.trim().slice(0, 100).replace(/[%_]/g, "")}%`;
  GO TO L4
L4: BEGIN transaction
  GO TO L6
L5: COMMIT transaction
  GO TO L2
L6: await event(tx, actor, "search", "patient");
  GO TO L7
L7: RETURN rows( sql`SELECT
id,mrn,first_name,last_name,dob,phone,address,gender,provisional_reason,merged_into,archived,version
FROM patients WHERE (mrn
  GO TO L5
```

patientDetail

Read a patient and visits. Add clinical history and documents only for a doctor.

Source: src/lib/service.ts. Graph: patientDetail.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START patientDetail
  GO TO L3
L2: END PROCEDURE
L3: idSchema.parse(id);
  GO TO L4
L4: BEGIN transaction
  GO TO L6
L5: COMMIT transaction
  GO TO L2
L6: const p = await found( await tx.select().from(patients).where(eq(patients.id, id)), );
  GO TO L7
L7: await event(tx, actor, "view", "patient", id);
  GO TO L8
L8: const visitList = await rows( sql`SELECT
v.id,v.patient_id,v.doctor_id,v.status,v.revision,v.visit_at,v.filed_at,u.name AS doctor_name FROM
visits v J
  GO TO L9
L9: const appointmentList = await rows( sql`SELECT a.*,u.name AS doctor_name FROM appointments a JOIN
users u ON u.id=a.doctor_id WHERE patient_id=${id} O
  GO TO L10
L10: IF actor.role !== "doctor"
  THEN GO TO L11
  ELSE GO TO L12
L11: RETURN { patient: p, visits: visitList, appointments: appointmentList }
  GO TO L5
L12: const clinical = await rows( sql`SELECT c.*,u.name AS author_name FROM clinical_revisions c JOIN
users u ON u.id=c.author_id JOIN visits v ON v.id=c.v
  GO TO L13
L13: const files = await tx .select({ id: documents.id, title: documents.title, kind: documents.kind,
mime: documents.mime, size: documents.size, createdAt
  GO TO L14
L14: RETURN { patient: p, visits: visitList, appointments: appointmentList, clinical, documents:
files, }
  GO TO L5
```

savePatient

Validate demographics. Create an MRN or update the expected record version. Warn about possible duplicates.

Source: src/lib/service.ts. Graph: savePatient.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START savePatient
  GO TO L3
L2: END PROCEDURE
L3: const data = patientInput.parse(input);
  GO TO L4
L4: BEGIN transaction
  GO TO L6
L5: COMMIT transaction
  GO TO L2
L6: IF id
  THEN GO TO L7
  ELSE GO TO L14
L7: idSchema.parse(id);
  GO TO L8
L8: const version = z .object({ version: z.number().int().positive() }) .parse(input).version;
  GO TO L9
L9: const [p] = await tx .update(patients) .set({ ...data, version: version + 1 })
.where(and(eq(patients.id, id), eq(patients.version, version))) .return
  GO TO L10
L10: IF p
  THEN GO TO L12
  ELSE GO TO L11
L11: RAISE new AppError( 409, "This patient changed. Reload the record before you save.", )
  GO TO L2
L12: await event(tx, actor, "update", "patient", id, { version });
  GO TO L13
L13: RETURN { patient: p }
  GO TO L5
L14: const duplicates = await rows( sql`SELECT id,mrn FROM patients WHERE
lower(first_name)=lower(${data.firstName}) AND lower(last_name)=lower(${data.last
  GO TO L15
L15: const [p] = await tx .insert(patients) .values({ ...data, mrn: sql`DEFAULT` }) .returning();
  GO TO L16
L16: await event(tx, actor, "create", "patient", p.id);
  GO TO L17
L17: RETURN { patient: p, duplicates }
  GO TO L5
```

archivePatient

Require a doctor and the current version before changing patient archive status.

Source: src/lib/service.ts. Graph: archivePatient.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START archivePatient
  GO TO L3
L2: END PROCEDURE
L3: await requireRole(actor);
  GO TO L4
L4: const { version, archived } = z .object({ version: z.number().int(), archived: z.boolean() })
.parse(input);
  GO TO L5
L5: BEGIN transaction
  GO TO L7
```

```
L6: COMMIT transaction
  GO TO L2
L7: const [p] = await tx .update(patients) .set({ archived, version: version + 1 }) .where(
and(eq(patients.id, idSchema.parse(id)), eq(patients.version,
  GO TO L8
L8: IF p
  THEN GO TO L10
  ELSE GO TO L9
L9: RAISE new AppError(409, "This record changed. Reload it.")
  GO TO L2
L10: IF archived
  THEN GO TO L11
  ELSE GO TO L13
L11: await event(tx, actor, archived ? "archive" : "restore", "patient", id);
  GO TO L12
L12: RETURN p
  GO TO L6
L13: Follow False branch
  GO TO L11
```

reconcilePatient

Require a doctor. Lock both records and link an unresolved provisional record to its primary record.

Source: src/lib/service.ts. Graph: reconcilePatient.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START reconcilePatient
  GO TO L3
L2: END PROCEDURE
L3: await requireRole(actor);
  GO TO L4
L4: const { targetId, reason } = z .object({ targetId: idSchema, reason:
z.string().trim().min(3).max(500) }) .parse(input);
  GO TO L5
L5: idSchema.parse(id);
  GO TO L6
L6: IF id === targetId
  THEN GO TO L7
  ELSE GO TO L8
L7: RAISE new AppError(400, "Choose a different patient.")
  GO TO L2
L8: BEGIN transaction
  GO TO L10
L9: COMMIT transaction
  GO TO L2
L10: const list = await rows( sql`SELECT * FROM patients WHERE id IN(${id}::uuid,${targetId}::uuid)
ORDER BY id FOR UPDATE`, tx, );
  GO TO L11
L11: IF list.length !== 2
  THEN GO TO L12
  ELSE GO TO L13
L12: RAISE new AppError(404, "Both patients must exist.")
  GO TO L2
L13: const source = list.find((p) => p.id === id);
  GO TO L14
L14: const target = list.find((p) => p.id === targetId);
  GO TO L15
L15: IF source?.provisional_reason
  THEN GO TO L16
  ELSE GO TO L18
L16: IF source.merged_into
  THEN GO TO L18
  ELSE GO TO L17
```

```
L17: IF target?.merged_into
  THEN GO TO L18
  ELSE GO TO L19
L18: RAISE new AppError( 400, "Use an unresolved provisional record and an active primary record.", )
  GO TO L2
L19: await tx .update(patients) .set({ mergedInto: targetId, archived: true, version:
Number(source.version) + 1, }) .where(eq(patients.id, id));
  GO TO L20

L20: await event(tx, actor, "reconcile", "patient", id, { targetId, reason });
  GO TO L21
L21: RETURN { targetId }
  GO TO L9
```

schedule

Read the appointments for one Jamaica calendar day and record the view.

Source: src/lib/service.ts. Graph: schedule.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START schedule
  GO TO L3
L2: END PROCEDURE
L3: IF date
  THEN GO TO L4
  ELSE GO TO L5
L4: IF /^\d{4}-\d{2}-\d{2}$/.test(date)
  THEN GO TO L5
  ELSE GO TO L10
L5: const day = date && /^\d{4}-\d{2}-\d{2}$/.test(date) ? date : new Intl.DateTimeFormat("en-CA", {
timeZone: "America/Jamaica", }).format(new Date());
  GO TO L6
L6: BEGIN transaction
  GO TO L8
L7: COMMIT transaction
  GO TO L2
L8: await event(tx, actor, "view", "schedule");
  GO TO L9
L9: RETURN rows( sql`SELECT a.*,p.mrn,p.first_name,p.last_name,u.name AS doctor_name FROM appointments
 a JOIN patients p ON p.id=a.patient_id JOIN users u
  GO TO L7
L10: Follow False branch
  GO TO L5
```

availability

Find future half-hour start slots inside doctor hours and outside bookings and blocked periods.

Source: src/lib/service.ts. Graph: availability.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START availability
  GO TO L3
L2: END PROCEDURE
L3: idSchema.parse(doctorId);
```

```
  GO TO L4
L4: IF /^\d{4}-\d{2}-\d{2}$/.test(date)
  THEN GO TO L6
  ELSE GO TO L5
L5: RAISE new AppError(400, "Choose a valid date.")
  GO TO L2
L6: await event(db, actor, "view", "availability");
  GO TO L7
L7: RETURN rows( sql`SELECT slot AS start_at,slot+interval '30 minutes' AS end_at FROM doctor_hours h
CROSS JOIN LATERAL generate_series((${date}::date+ma
  GO TO L2
```

bookAppointment

Validate the interval and active patient. Insert a confirmed booking or update a current confirmed booking. Audit only a committed change.

Source: src/lib/service.ts. Graph: bookAppointment.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START bookAppointment
  GO TO L3
L2: END PROCEDURE
L3: const data = appointmentInput.parse(input);
  GO TO L4
L4: BEGIN transaction
  GO TO L6
L5: COMMIT transaction
  GO TO L2
L6: const p = await found( await tx.select().from(patients).where(eq(patients.id, data.patientId)), );
  GO TO L7
L7: IF p.archived
  THEN GO TO L9
  ELSE GO TO L8
L8: IF p.mergedInto
  THEN GO TO L9
  ELSE GO TO L10
L9: RAISE new AppError(400, "Choose an active patient record.")
  GO TO L2
L10: IF id
  THEN GO TO L11
  ELSE GO TO L18
L11: idSchema.parse(id);
  GO TO L12
L12: const { version } = z .object({ version: z.number().int().positive() }) .parse(input);
  GO TO L13
L13: const [a] = await tx .update(appointments) .set({ ...data, startAt: new Date(data.startAt),
endAt: new Date(data.endAt), version: version + 1, }) .whe
  GO TO L14
L14: IF a
  THEN GO TO L16
  ELSE GO TO L15
L15: RAISE new AppError( 409, "This appointment changed or has started. Reload the schedule.", )
  GO TO L2
L16: await event(tx, actor, "reschedule", "appointment", id, { version });
  GO TO L17
L17: RETURN a
  GO TO L5
L18: const [a] = await tx .insert(appointments) .values({ ...data, startAt: new Date(data.startAt),
endAt: new Date(data.endAt), }) .returning();
  GO TO L19
L19: await event(tx, actor, "confirm", "appointment", a.id);
  GO TO L20
L20: RETURN a
```

```
  GO TO L5
```

appointmentStatus

Change a current confirmed appointment to arrived or cancelled. Preserve the booking record.

Source: src/lib/service.ts. Graph: appointmentStatus.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START appointmentStatus
  GO TO L3
L2: END PROCEDURE
L3: const { version, status } = z .object({ version: z.number().int(), status: z.enum(["cancelled",
"arrived"]), }) .parse(input);
  GO TO L4
L4: BEGIN transaction
  GO TO L6
L5: COMMIT transaction
  GO TO L2
L6: const [a] = await tx .update(appointments) .set({ status, version: version + 1 }) .where( and(
eq(appointments.id, idSchema.parse(id)), eq(appointment
  GO TO L7
L7: IF a
  THEN GO TO L9
  ELSE GO TO L8
L8: RAISE new AppError( 409, "This appointment changed or cannot enter that state.", )
  GO TO L2
L9: await event(tx, actor, status, "appointment", id);
  GO TO L10
L10: RETURN a
  GO TO L5
```

startVisit

Create a walk-in visit or start a matching appointment. Keep its doctor and patient association.

Source: src/lib/service.ts. Graph: startVisit.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START startVisit
  GO TO L3
L2: END PROCEDURE
L3: const data = z .object({ patientId: idSchema, doctorId: idSchema, appointmentId:
idSchema.optional(), }) .parse(input);
  GO TO L4
L4: BEGIN transaction
  GO TO L6
L5: COMMIT transaction
  GO TO L2
L6: const p = await found( await tx.select().from(patients).where(eq(patients.id, data.patientId)), );
  GO TO L7
L7: IF p.archived
  THEN GO TO L8
  ELSE GO TO L9
L8: RAISE new AppError(400, "This patient record is archived.")
  GO TO L2
```

```
L9: await found( await rows( sql`SELECT id FROM users WHERE id=${data.doctorId}::uuid AND active AND
role='doctor'`, tx, ), "Choose an active doctor.", );
  GO TO L10
L10: IF data.appointmentId
  THEN GO TO L11
  ELSE GO TO L17
L11: const a = await found( await rows( sql`SELECT * FROM appointments WHERE
id=${data.appointmentId}::uuid FOR UPDATE`, tx, ), );
  GO TO L12
L12: IF a.patient_id !== data.patientId
  THEN GO TO L15
  ELSE GO TO L13
L13: IF a.doctor_id !== data.doctorId
  THEN GO TO L15
  ELSE GO TO L14
L14: IF ["confirmed", "arrived"].includes(String(a.status))
  THEN GO TO L16
  ELSE GO TO L15
L15: RAISE new AppError( 400, "Use a matching confirmed or arrived appointment.", )
  GO TO L2
L16: await tx .update(appointments) .set({ status: "in_consultation", version: Number(a.version) + 1
}) .where(eq(appointments.id, data.appointmentId));
  GO TO L17
L17: const [v] = await tx.insert(visits).values(data).returning();
  GO TO L18
L18: await event(tx, actor, "start", "visit", v.id);
  GO TO L19
L19: RETURN v

  GO TO L5
```

saveClinical

Require doctor access. Lock the visit, check its revision, and append a draft, final entry, or reasoned final correction.

Source: src/lib/service.ts. Graph: saveClinical.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START saveClinical
  GO TO L3
L2: END PROCEDURE
L3: await requireRole(actor);
  GO TO L4
L4: idSchema.parse(id);
  GO TO L5
L5: const data = clinicalInput.parse(input);
  GO TO L6
L6: BEGIN transaction
  GO TO L8
L7: COMMIT transaction
  GO TO L2
L8: const v = await found( await rows(sql`SELECT * FROM visits WHERE id=${id}::uuid FOR UPDATE`, tx),
);
  GO TO L9
L9: IF Number(v.revision) !== data.expectedRevision
  THEN GO TO L10
  ELSE GO TO L11
L10: RAISE new AppError( 409, "Another doctor saved a revision. Reload this visit.", )
  GO TO L2
L11: const previous = await rows( sql`SELECT state FROM clinical_revisions WHERE visit_id=${id}::uuid
ORDER BY revision DESC LIMIT 1`, tx, );
  GO TO L12
L12: IF previous[0]?.state === "final"
```

```
  THEN GO TO L13
  ELSE GO TO L16
L13: IF data.reason
  THEN GO TO L14
  ELSE GO TO L15
L14: IF data.state !== "final"
  THEN GO TO L15
  ELSE GO TO L16
L15: RAISE new AppError( 400, "Add a correction reason and save a final correction.", )
  GO TO L2
L16: const { expectedRevision, state, reason, ...content } = data;
  GO TO L17
L17: const revision = expectedRevision + 1;
  GO TO L18
L18: const [r] = await tx .insert(clinicalRevisions) .values({ visitId: id, revision, authorId:
actor.id, state, reason, content, }) .returning();
  GO TO L19
L19: IF v.status === "filed"
  THEN GO TO L21
  ELSE GO TO L20

L20: IF state === "final"
  THEN GO TO L21
  ELSE GO TO L28
L21: await tx .update(visits) .set({ revision, status: v.status === "filed" ? "filed" : state ===
"final" ? "ready" : "open", }) .where(eq(visits.id, id));
  GO TO L22
L22: IF state === "final"
  THEN GO TO L23
  ELSE GO TO L25
L23: IF More items in content.medications
  THEN GO TO L24
  ELSE GO TO L25
L24: await tx.execute( sql`INSERT INTO prescriptions(visit_id,revision_id,patient_id,content)
VALUES(${id},${r.id},${v.patient_id}::uuid,${JSON.stringify(m
  GO TO L23
L25: IF previous[0]?.state === "final"
  THEN GO TO L26
  ELSE GO TO L29
L26: await event( tx, actor, previous[0]?.state === "final" ? "correct" : "save_revision", "visit",
id, { revision, state, reason }, );
  GO TO L27
L27: RETURN r
  GO TO L7
L28: Follow False branch
  GO TO L21
L29: Follow False branch
  GO TO L26
```

fileVisit

Lock the visit. Require the current final diagnosis. File the visit and complete its appointment.

Source: src/lib/service.ts. Graph: fileVisit.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START fileVisit
  GO TO L3
L2: END PROCEDURE
L3: const { expectedRevision } = z .object({ expectedRevision: z.number().int().positive() })
.parse(input);
  GO TO L4
L4: BEGIN transaction
  GO TO L6
```

```
L5: COMMIT transaction
  GO TO L2
L6: const v = await found( await rows( sql`SELECT * FROM visits WHERE id=${idSchema.parse(id)}::uuid
FOR UPDATE`, tx, ), );
  GO TO L7
L7: const c = await rows( sql`SELECT content,state FROM clinical_revisions WHERE visit_id=${id}::uuid
AND revision=${expectedRevision}`, tx, );
  GO TO L8
L8: IF v.status !== "ready"
  THEN GO TO L12
  ELSE GO TO L9
L9: IF Number(v.revision) !== expectedRevision
  THEN GO TO L12
  ELSE GO TO L10
L10: IF c[0]?.state !== "final"
  THEN GO TO L12
  ELSE GO TO L11
L11: IF (c[0].content as Row)?.diagnosis
  THEN GO TO L13
  ELSE GO TO L12
L12: RAISE new AppError( 409, "The doctor must complete the diagnosis before this visit can be
filed.", )
  GO TO L2
L13: await tx .update(visits) .set({ status: "filed", filedAt: new Date() }) .where(eq(visits.id,
id));
  GO TO L14
L14: IF v.appointment_id
  THEN GO TO L15
  ELSE GO TO L16
L15: await tx.execute( sql`UPDATE appointments SET status='completed',version=version+1 WHERE
id=${v.appointment_id}::uuid`, );
  GO TO L16
L16: await event(tx, actor, "file", "visit", id);
  GO TO L17
L17: RETURN { filed: true }
  GO TO L5
```

saveInvoice

Validate service lines and calculate integer-cent totals before creating an invoice draft.

Source: src/lib/service.ts. Graph: saveInvoice.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START saveInvoice
  GO TO L3
L2: END PROCEDURE
L3: const data = invoiceInput.parse(input);
  GO TO L4
L4: const totalCents = invoiceTotal(data.lines);
  GO TO L5
L5: BEGIN transaction
  GO TO L7
L6: COMMIT transaction
  GO TO L2
L7: await found( await tx .select({ id: patients.id }) .from(patients) .where(eq(patients.id,
data.patientId)), );
  GO TO L8
L8: const [i] = await tx .insert(invoices) .values({ ...data, totalCents, number: sql`DEFAULT` })
.returning();
  GO TO L9
L9: await event(tx, actor, "create", "invoice", i.id);
  GO TO L10
L10: RETURN i
```

```
  GO TO L6
```

setInvoiceStatus

Issue a draft or void an issued invoice with a reason. Reject stale versions.

Source: src/lib/service.ts. Graph: setInvoiceStatus.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START setInvoiceStatus
  GO TO L3
L2: END PROCEDURE
L3: const { version, status, reason } = z .object({ version: z.number().int(), status:
z.enum(["issued", "void"]), reason: z.string().trim().max(500).defa
  GO TO L4
L4: IF status === "void"
  THEN GO TO L5
  ELSE GO TO L7
L5: IF reason
  THEN GO TO L7
  ELSE GO TO L6
L6: RAISE new AppError(400, "Enter a reason to void the invoice.")
  GO TO L2
L7: BEGIN transaction
  GO TO L9
L8: COMMIT transaction
  GO TO L2
L9: IF status === "issued"
  THEN GO TO L10
  ELSE GO TO L15
L10: const [i] = await tx .update(invoices) .set({ status, reason, version: version + 1 }) .where(
and( eq(invoices.id, idSchema.parse(id)), eq(invoices.ve
  GO TO L11
L11: IF i
  THEN GO TO L13
  ELSE GO TO L12
L12: RAISE new AppError( 409, "This invoice changed or cannot enter that state.", )
  GO TO L2
L13: await event(tx, actor, status, "invoice", id, { reason });
  GO TO L14
L14: RETURN i
  GO TO L8
L15: Follow False branch
  GO TO L10
```

invoiceList

Read administrative invoice data and record the view.

Source: src/lib/service.ts. Graph: invoiceList.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START invoiceList
  GO TO L3
L2: END PROCEDURE
L3: await event(db, actor, "view", "invoice_list");
```

```
  GO TO L4
L4: RETURN rows( sql`SELECT i.*,p.first_name,p.last_name,p.mrn FROM invoices i JOIN patients p ON
p.id=i.patient_id ORDER BY i.created_at DESC LIMIT 100`,
  GO TO L2
```

manageUser

Require doctor access. Create or change an account. Protect demo accounts, keep a doctor active, and revoke changed sessions.

Source: src/lib/service.ts. Graph: manageUser.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START manageUser
  GO TO L3
L2: END PROCEDURE
L3: await requireRole(actor);
  GO TO L4
L4: IF id
  THEN GO TO L14
  ELSE GO TO L5
L5: const d = userInput.parse(input);
  GO TO L6
L6: const passwordHash = await hashPassword(d.password);
  GO TO L7
L7: BEGIN transaction
  GO TO L9
L8: COMMIT transaction
  GO TO L2
L9: const [u] = await tx .insert(users) .values({ username: d.username, name: d.name, role: d.role,
passwordHash, }) .returning({ id: users.id });
  GO TO L10
L10: IF d.role === "doctor"
  THEN GO TO L11
  ELSE GO TO L12
L11: await tx.execute( sql`INSERT INTO doctor_hours(doctor_id,weekday,start_min,end_min) SELECT
${u.id}::uuid,n,480,1020 FROM generate_series(1,5)n`, );
  GO TO L12
L12: await event(tx, actor, "create", "user", u.id);
  GO TO L13
L13: RETURN u
  GO TO L8
L14: const d = z .object({ role: z.enum(["doctor", "front_desk"]), active: z.boolean(), version:
z.number().int(), password: z.string().min(12).max(128).op
  GO TO L15
L15: IF d.password
  THEN GO TO L16
  ELSE GO TO L32
L16: const passwordHash = d.password ? await hashPassword(d.password) : undefined;
  GO TO L17
L17: BEGIN transaction
  GO TO L19
L18: COMMIT transaction
  GO TO L2
L19: const old = await found( await tx .select() .from(users) .where(eq(users.id,
idSchema.parse(id))), );
  GO TO L20
L20: IF old.protected

  THEN GO TO L21
  ELSE GO TO L23
L21: IF process.env.DEMO_MODE === "true"
  THEN GO TO L22
  ELSE GO TO L23
```

```
L22: RAISE new AppError(403, "The supplied demo accounts cannot be changed.")
  GO TO L2
L23: IF passwordHash
  THEN GO TO L24
  ELSE GO TO L33
L24: const [u] = await tx .update(users) .set({ role: d.role, active: d.active, version: d.version +
1, ...(passwordHash ? { passwordHash } : {}), }) .wher
  GO TO L25
L25: IF u
  THEN GO TO L27
  ELSE GO TO L26
L26: RAISE new AppError(409, "This account changed. Reload the list.")
  GO TO L2
L27: IF d.role === "doctor"
  THEN GO TO L28
  ELSE GO TO L29
L28: await tx.execute( sql`INSERT INTO doctor_hours(doctor_id,weekday,start_min,end_min) SELECT
${id}::uuid,n,480,1020 FROM generate_series(1,5)n ON CONFLI
  GO TO L29
L29: await tx.delete(sessions).where(eq(sessions.userId, id));
  GO TO L30
L30: await event(tx, actor, "update", "user", id);
  GO TO L31
L31: RETURN u
  GO TO L18
L32: Follow False branch
  GO TO L16
L33: Follow False branch
  GO TO L24
```

changePassword

Verify the current password, hash the replacement, and revoke existing sessions.

Source: src/lib/service.ts. Graph: changePassword.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START changePassword
  GO TO L3
L2: END PROCEDURE
L3: const { currentPassword, password } = z .object({ currentPassword: z.string().max(128), password:
z.string().min(12).max(128), }) .parse(input);
  GO TO L4
L4: IF actor.protected
  THEN GO TO L5
  ELSE GO TO L7
L5: IF process.env.DEMO_MODE === "true"
  THEN GO TO L6
  ELSE GO TO L7
L6: RAISE new AppError(403, "The supplied demo password cannot be changed.")
  GO TO L2
L7: const [u] = await db.select().from(users).where(eq(users.id, actor.id));
  GO TO L8
L8: const { verify } = await import("@node-rs/argon2");
  GO TO L9
L9: IF await verify(u.passwordHash, currentPassword)
  THEN GO TO L11
  ELSE GO TO L10
L10: RAISE new AppError(400, "The current password is incorrect.")
  GO TO L2
L11: const passwordHash = await hashPassword(password);
  GO TO L12
L12: BEGIN transaction
  GO TO L14
```

```
L13: COMMIT transaction
  GO TO L17
L14: await tx .update(users) .set({ passwordHash, version: u.version + 1 }) .where(eq(users.id,
actor.id));
  GO TO L15
L15: await tx.delete(sessions).where(eq(sessions.userId, actor.id));
  GO TO L16
L16: await event(tx, actor, "password_change", "user", actor.id);
  GO TO L13
L17: RETURN { logout: true }
  GO TO L2
```

dashboard

Read practice counts, ready visits, and in-app reminders without clinical note bodies.

Source: src/lib/service.ts. Graph: dashboard.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START dashboard
  GO TO L3
L2: END PROCEDURE
L3: BEGIN transaction
  GO TO L5
L4: COMMIT transaction
  GO TO L2
L5: await event(tx, actor, "view", "dashboard");
  GO TO L6
L6: const counts = await rows( sql`SELECT (SELECT count(*) FROM patients WHERE NOT archived) AS
patients,(SELECT count(*) FROM appointments WHERE status I
  GO TO L7
L7: const ready = await rows( sql`SELECT
v.id,v.revision,v.status,v.patient_id,p.first_name,p.last_name,p.mrn FROM visits v JOIN patients p ON
p.id=v.pati
  GO TO L8
L8: const reminders = await rows( sql`SELECT r.id,a.start_at,p.first_name,p.last_name,p.mrn FROM
reminders r JOIN appointments a ON a.id=r.appointment_id
  GO TO L9
L9: RETURN { counts: counts[0], ready, reminders }
  GO TO L4
```

reportData

Select the report type, require clinical permission where needed, and record an export event.

Source: src/lib/service.ts. Graph: reportData.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START reportData
  GO TO L3
L2: END PROCEDURE
L3: IF [ "schedule", "attendance", "invoice", "history", "prescription", "certificate", "summary",
"visits", ].includes(type)
  THEN GO TO L5
  ELSE GO TO L4
L4: RAISE new AppError(404, "The report was not found.")
```

```
  GO TO L2
L5: IF ["history", "prescription", "certificate", "summary"].includes(type)
  THEN GO TO L6
  ELSE GO TO L7
L6: await requireRole(actor);
  GO TO L7
L7: IF type === "schedule"
  THEN GO TO L8
  ELSE GO TO L9
L8: RETURN { type, appointments: await schedule(actor, id) }
  GO TO L2
L9: IF type === "visits"
  THEN GO TO L10
  ELSE GO TO L12
L10: await event(db, actor, "export", "visit_counts");
  GO TO L11
L11: RETURN { type, counts: await rows( sql`SELECT (visit_at AT TIME ZONE 'America/Jamaica')::date AS
day,count(*) AS visits,count(*) FILTER(WHERE status='
  GO TO L2
L12: idSchema.parse(id);
  GO TO L13
L13: await event(db, actor, "export", type, id);
  GO TO L14
L14: IF type === "invoice"
  THEN GO TO L15
  ELSE GO TO L16
L15: RETURN { type, invoice: await found( await rows( sql`SELECT
i.*,p.first_name,p.last_name,p.mrn,p.address FROM invoices i JOIN patients p ON p.id=i.pat
  GO TO L2
L16: IF type === "history"
  THEN GO TO L17
  ELSE GO TO L18
L17: RETURN { type, ...(await patientDetail(actor, id)) }
  GO TO L2
L18: const v = await found( await rows( sql`SELECT v.*,p.first_name,p.last_name,p.mrn,u.name AS
doctor_name FROM visits v JOIN patients p ON p.id=v.patient
  GO TO L19

L19: IF type === "attendance"
  THEN GO TO L20
  ELSE GO TO L21
L20: RETURN { type, visit: { id: v.id, mrn: v.mrn, first_name: v.first_name, last_name: v.last_name,
visit_at: v.visit_at, doctor_name: v.doctor_name, stat
  GO TO L2
L21: const c = await found( await rows( sql`SELECT content,state,revision,created_at FROM
clinical_revisions WHERE visit_id=${id}::uuid ORDER BY revision D
  GO TO L22
L22: IF c.state !== "final"
  THEN GO TO L23
  ELSE GO TO L24
L23: RAISE new AppError( 400, "Complete the clinical entry before you print this document.", )
  GO TO L2
L24: RETURN { type, visit: v, clinical: c }
  GO TO L2
```

requireRole

Check the server account role and audit denied access.

Source: src/lib/auth.ts. Graph: requireRole.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START requireRole
  GO TO L3
```

```
L2: END PROCEDURE
L3: IF actor.role !== role
  THEN GO TO L4
  ELSE GO TO L2
L4: await auditDenied(actor, "clinical_or_admin");
  GO TO L5
L5: RAISE new AppError(403, "Your account cannot use this function.")
  GO TO L2
```

getSession

Hash the cookie token and find an unexpired session for an active account.

Source: src/lib/auth.ts. Graph: getSession.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START getSession
  GO TO L3
L2: END PROCEDURE
L3: IF token
  THEN GO TO L5
  ELSE GO TO L4
L4: RETURN null
  GO TO L2
L5: const rows = await db .select({ id: users.id, username: users.username, name: users.name, role:
users.role, protected: users.protected, version: users
  GO TO L6
L6: IF rows[0] is not null
  THEN GO TO L7
  ELSE GO TO L8
L7: RETURN rows[0] ?? null
  GO TO L2
L8: Follow False branch
  GO TO L7
```

login

Limit attempts, verify a password hash, create a random hashed session, and record the result.

Source: src/lib/auth.ts. Graph: login.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START login
  GO TO L3
L2: END PROCEDURE
L3: const key = digest(username.toLowerCase().trim() + ":" + ip);
  GO TO L4
L4: const attempt = await db.execute( sql`INSERT INTO login_attempts(key,count,expires_at)
VALUES(${key},1,now()+interval '15 minutes') ON CONFLICT(key) D
  GO TO L5
L5: IF Number(attempt.rows[0].count) > 8
  THEN GO TO L6
  ELSE GO TO L7
L6: RAISE new AppError( 429, "Too many login attempts. Try again in 15 minutes.", )
  GO TO L2
L7: const [u] = await db .select() .from(users) .where(eq(users.username,
```

```
username.toLowerCase().trim())) .limit(1);
  GO TO L8
L8: dummy ??= hashPassword(randomBytes(32).toString("hex"));
  GO TO L9
L9: IF u?.passwordHash is not null
  THEN GO TO L10
  ELSE GO TO L23
L10: const valid = await verify(u?.passwordHash ?? (await dummy), password).catch( () => false, );
  GO TO L11
L11: IF u
  THEN GO TO L12
  ELSE GO TO L14
L12: IF u.active
  THEN GO TO L13
  ELSE GO TO L14
L13: IF valid
  THEN GO TO L16
  ELSE GO TO L14
L14: await db .insert(auditEvents) .values({ action: "login_failed", entity: "session" });
  GO TO L15
L15: RAISE new AppError(401, "The username or password is incorrect.")
  GO TO L2
L16: const token = randomBytes(32).toString("hex");
  GO TO L17
L17: BEGIN transaction
  GO TO L19
L18: COMMIT transaction
  GO TO L22
L19: await tx .insert(sessions) .values({ id: digest(token), userId: u.id, expiresAt: new
Date(Date.now() + 8 * 3600000), });
  GO TO L20

L20: await tx.execute(sql`DELETE FROM login_attempts WHERE key=${key}`);
  GO TO L21
L21: await tx .insert(auditEvents) .values({ actorId: u.id, action: "login", entity: "session" });
  GO TO L18
L22: RETURN { token, user: { id: u.id, name: u.name, role: u.role, username: u.username }, }
  GO TO L2
L23: Follow False branch
  GO TO L10
```

storeFile

Validate length, signature, and public demo sample rules before writing an immutable private object.

Source: src/lib/storage.ts. Graph: storeFile.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START storeFile
  GO TO L3
L2: END PROCEDURE
L3: IF bytes.length < 1
  THEN GO TO L5
  ELSE GO TO L4
L4: IF bytes.length > 4194304
  THEN GO TO L5
  ELSE GO TO L6
L5: RAISE new AppError(400, "Use a file from 1 byte to 4 MiB.")
  GO TO L2
L6: const mime = fileMime(bytes);
  GO TO L7
L7: IF mime
  THEN GO TO L8
  ELSE GO TO L9
```

```
L8: IF mime !== claimedMime
  THEN GO TO L9
  ELSE GO TO L10
L9: RAISE new AppError( 400, "Use a PDF, PNG, or JPEG file with a matching file type.", )
  GO TO L2
L10: IF process.env.DEMO_MODE === "true"
  THEN GO TO L11
  ELSE GO TO L13
L11: IF bytes.equals(samplePdf)
  THEN GO TO L13
  ELSE GO TO L12
L12: RAISE new AppError( 400, "The public demo accepts only the supplied sample report.", )
  GO TO L2
L13: IF process.env.VERCEL_ENV === "preview"
  THEN GO TO L14
  ELSE GO TO L22
L14: const pathname = `practicechart/${process.env.VERCEL_ENV === "preview" ? "preview" :
"records"}/${randomUUID()}`;
  GO TO L15
L15: IF process.env.STORAGE_DRIVER === "blob"
  THEN GO TO L16
  ELSE GO TO L17
L16: await put(pathname, bytes, { access: "private", contentType: mime, addRandomSuffix: false, });
  GO TO L21
L17: IF process.env.PRIVATE_STORAGE_DIR
  THEN GO TO L18
  ELSE GO TO L23
L18: const root = path.resolve( /* turbopackIgnore: true */ process.env.PRIVATE_STORAGE_DIR ||
".private/uploads", );

  GO TO L19
L19: await mkdir(path.dirname(path.join(root, pathname)), { recursive: true });
  GO TO L20
L20: await writeFile(path.join(root, pathname), bytes, { flag: "wx" });
  GO TO L21
L21: RETURN { pathname, mime, size: bytes.length, sha256:
createHash("sha256").update(bytes).digest("hex"), }
  GO TO L2
L22: Follow False branch
  GO TO L14
L23: Follow False branch
  GO TO L18
```

readPrivateFile

Reject unsafe paths and retrieve a private object. Return a clear missing-file error.

Source: src/lib/storage.ts. Graph: readPrivateFile.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START readPrivateFile
  GO TO L3
L2: END PROCEDURE
L3: IF /^practicechart\/(records|preview)\/[a-f0-9-]{36}$/.test(pathname)
  THEN GO TO L5
  ELSE GO TO L4
L4: RAISE new AppError(404, "The file was not found.")
  GO TO L2
L5: IF I/O operation succeeds
  THEN GO TO L7
  ELSE GO TO L6
L6: Handle I/O failure
  GO TO L15
L7: IF process.env.STORAGE_DRIVER === "blob"
```

```
  THEN GO TO L8
  ELSE GO TO L13
L8: const r = await get(pathname, { access: "private" });
  GO TO L9
L9: IF r
  THEN GO TO L10
  ELSE GO TO L11
L10: IF r.statusCode !== 200
  THEN GO TO L11
  ELSE GO TO L12
L11: RAISE new Error("Missing")
  GO TO L6
L12: RETURN Buffer.from(await new Response(r.stream).arrayBuffer())
  GO TO L2
L13: IF process.env.PRIVATE_STORAGE_DIR
  THEN GO TO L14
  ELSE GO TO L16
L14: RETURN await readFile( /* turbopackIgnore: true */ path.join( path.resolve( /* turbopackIgnore:
true */ process.env.PRIVATE_STORAGE_DIR || ".private/u
  GO TO L2
L15: RAISE new AppError( 404, "The stored file is not available. Contact the practice operator.", )
  GO TO L2
L16: Follow False branch
  GO TO L14
```

invoiceTotal

Sum integer quantity times unit cents and reject unsafe or excessive totals.

Source: src/lib/validation.ts. Graph: invoiceTotal.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START invoiceTotal
  GO TO L3
L2: END PROCEDURE
L3: const total = lines.reduce((n, l) => n + l.quantity * l.unitCents, 0);
  GO TO L4
L4: IF Number.isSafeInteger(total)
  THEN GO TO L5
  ELSE GO TO L6
L5: IF total > 2000000000
  THEN GO TO L6
  ELSE GO TO L7
L6: RAISE new Error("Invoice total is too large.")
  GO TO L2
L7: RETURN total
  GO TO L2
```

fileMime

Classify only PDF, PNG, or JPEG file signatures.

Source: src/lib/validation.ts. Graph: fileMime.

The following labelled pseudocode follows graph edges. Labels are not execution order. A callback RETURN sets its result; the COMMIT step must succeed before the service returns that result. A raised error ends the transaction with rollback. Long source operations are abbreviated here; use the indexed source listing for their full arguments.

```
L1: START fileMime
```

```
  GO TO L3
L2: END PROCEDURE
L3: IF bytes.subarray(0, 5).toString() === "%PDF-"
  THEN GO TO L4
  ELSE GO TO L5
L4: RETURN "application/pdf"
  GO TO L2
L5: IF bytes.subarray(0, 8).equals(Buffer.from([137, 80, 78, 71, 13, 10, 26, 10]))
  THEN GO TO L6
  ELSE GO TO L7
L6: RETURN "image/png"
  GO TO L2
L7: IF bytes[0] === 255
  THEN GO TO L8
  ELSE GO TO L11
L8: IF bytes[1] === 216
  THEN GO TO L9
  ELSE GO TO L11
L9: IF bytes[2] === 255
  THEN GO TO L10
  ELSE GO TO L11
L10: RETURN "image/jpeg"
  GO TO L2
L11: RETURN null
  GO TO L2
```